

more than 30 I/O pins, offering the flexibility to add displays, sensors, bulbs, relays, and actuators, all

controlled through a single system.

Its inbuilt touch-sensing I/O supports touch-based switching,

resulting in an AI assistant-enabled home automation system that is end-user ready, suitable for production, and fully customisable. Fig. 1 shows the prototype, and the required components are listed in the Bill of Materials table.

Software

ESP RainMaker is used as the software platform. The IndusBoard Coin runs custom RainMaker firmware, enabling devices within the system to be controlled, monitored, and updated through the RainMaker app. The system can be seamlessly linked to Alexa, Google Assistant, and Apple Home, enabling control through voice assistants.

The platform also supports remote monitoring, firmware updates, and sensor-based automation, while remaining fully customisable. Prebuilt firmware for light, fan, and switch control is available on the RainMaker platform; however, the platform focuses on a fully customisable implementation, with custom firmware developed in the Arduino IDE.

In the firmware design, the RainMaker library is used. As addressable LEDs are used for light colour and brightness control, an additional library, such as FastLED, is required to ensure efficient LED management.

Although a human-machine interface with a display-based interface can be implemented, this system focuses on a simplified design without a display. Instead, the touch I/O pins are used as touch switches. The system can be controlled via Alexa or Google Assistant voice commands, the mobile app, or the touch switches.

The board provides more than ten touch and proximity (contactless) I/O pins. A touch threshold is defined in the code to set the sensitivity of the touch inputs.

The provisioning name and proof of possession (PoP) can also be

```

51 static Switch* my_switch = nullptr;
52 static TemperatureSensor* my_temp = nullptr;
53
54 // Provisioning callback
55 void sysProvEvent(arduino_event_t *sys_event) {
56     switch (sys_event->event_id) {
57         case ARDUINO_EVENT_PROV_START:
58             #if CONFIG_IDF_TARGET_ESP32S2 || CONFIG_IDF_TARGET_ESP32S3
59                 Serial.printf("\nProvisioning SoftAP: %s | PoP: %s\n", service_name, pop);
60                 printQR(service_name, pop, "softap");
61             #else
62                 Serial.printf("\nProvisioning BLE: %s | PoP: %s\n", service_name, pop);
63                 printQR(service_name, pop, "ble");
64             #endif
65             break;
66         case ARDUINO_EVENT_PROV_INIT:
67             wifi_prov_mgr_disable_auto_stop(10000);
68             break;
69         case ARDUINO_EVENT_PROV_CRED_SUCCESS:
70             wifi_prov_mgr_stop_provisioning();
71             break;
72         default: break;
73     }
74 }
75
76 // Update functions
77 void updateFan() {
78     analogWrite(fan_pin, fan_power ? map(fan_speed, 1, 5, 80, 255) : 0);
79 }

```

Fig. 3: Code snippet showing the provisioning function

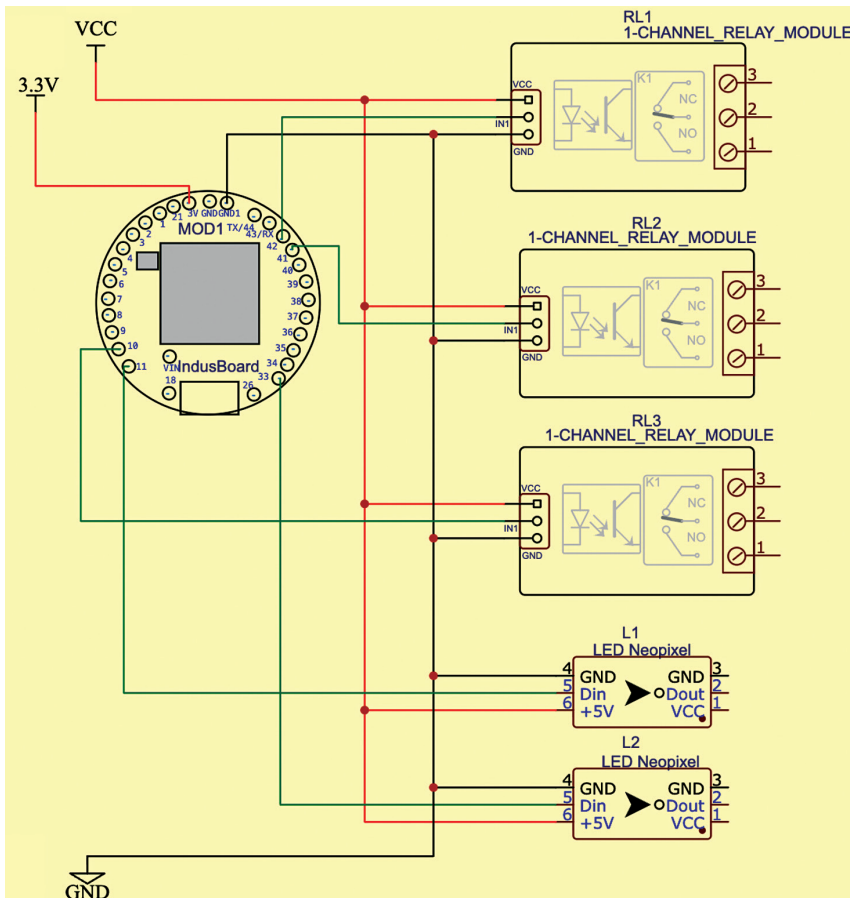


Fig. 4: Circuit diagram